

Disciplina: Sistemas Distribuído

Docente: Romilton Lima

3º Ano / 2º Semestre: LEIT

Projeto 2 — Sistemas Distribuídos

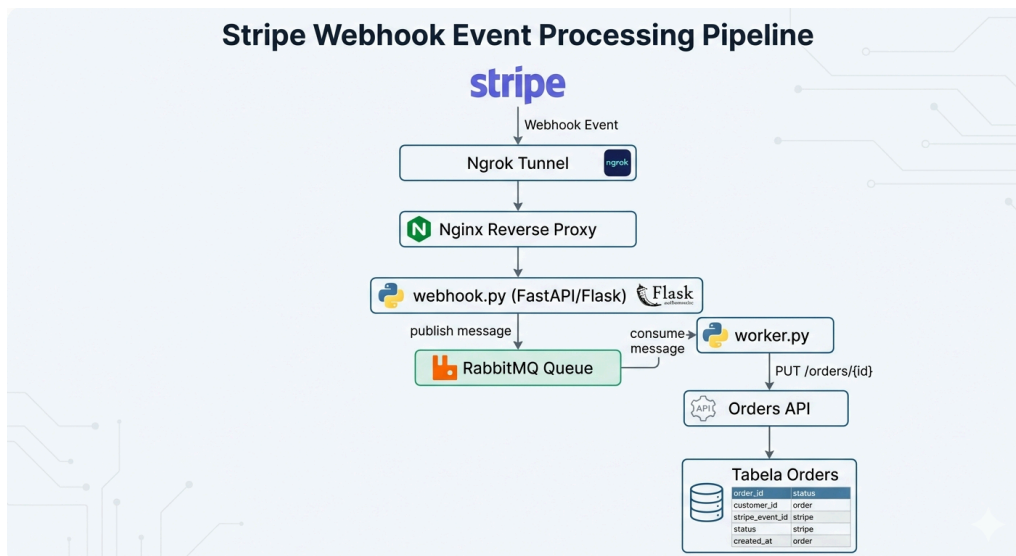
Integração Stripe Webhook + Ngrok + Nginx + RabbitMQ + API Orders (orders_service)

Objetivo

Implementar uma arquitetura distribuída baseada em eventos utilizando a plataforma de pagamentos Stripe, comunicação assíncrona com RabbitMQ, exposição pública via ngrok e roteamento com NGINX reverse proxy.

O sistema deverá receber eventos do Stripe através de Webhooks, encaminhar os eventos para um Message Broker RabbitMQ e atualizar o estado das encomendas (orders) na API de Orders (order_service) através de requisições HTTP PUT.

Arquitetura do Projeto



Requisitos Técnicos

1. Configuração do Ngrok Tunnel

Criar um tunnel público utilizando ngrok para expor o servidor local.

Exemplo:

ngrok http 8080

O endpoint HTTPS gerado deverá ser utilizado no Stripe Webhook Destination.

2. Configuração do Nginx Reverse Proxy

Configurar o Nginx para encaminhar os requests recebidos do ngrok para os microserviços internos.

Exemplo:

```
server {  
    listen 8080;  
    ...  
}
```

3. Configuração do Stripe Webhook Destination

No dashboard da Stripe:

- Criar um Webhook Destination
- Endpoint: **https://xxxx.ngrok-free.app/webhook**

Selecionar os eventos:

- **checkout.session.completed**
- **payment_intent.succeeded**
- **payment_intent.payment_failed**

4. Programação do webhook.py

O webhook deve:

- Receber eventos do Stripe
- Validar assinatura do webhook
- Extrair: **stripe_events**
- Publicar mensagens no RabbitMQ

Tecnologias sugeridas:

- Python
- FastAPI
- pika
- stripe SDK

5. Configuração do RabbitMQ

Criar:

- Exchange (**exchange=""**)
- Queue (**channel.queue_declare(queue='stripe_events')**)
- Binding (**routing_key='stripe_events'**)

Fila sugerida:

stripe_events

Painel: **http://localhost:15672**

Usuário: **guest**

Senha: **guest**

6. Programação do worker.py

O worker deve:

- Consumir mensagens do RabbitMQ
- Interpretar o evento Stripe
- Atualizar a API Orders (order_service) via HTTP PUT

Exemplo:

```
response = requests.put(
    f"http://localhost:9000/orders/{order_id}",
    json={
        "status": "PAID",
        "stripe_event_id": env_xxx
    }
)
```

Eventos esperados:

Evento Stripe	Estado da Order
payment_intent.succeeded	PAID

7. Atualização da API Orders

A API Orders deverá possuir endpoint PUT:

PUT /orders/{order_id}

Exemplo body:

```
{
    "status": "paid",
    "stripe_event_id": env_xxx
}
```

A tabela Orders deverá ser atualizada automaticamente após processamento do worker.

Estrutura Esperada do Projeto

webhook/

| _____ [webhook.py](#)
| _____ [worker.py](#)

order_service/

| _____ [database.py](#)
| _____ [main.py](#)
| _____ [models.py](#)

Para entregar

Os alunos deverão entregar:

1. Código fonte completo
2. README com instruções de execução

Critérios de Avaliação

Critério	Peso
Configuração correta do Ngrok	10%
Configuração do Nginx Reverse Proxy	5%
Configuração do Stripe Webhook	15%
Implementação do webhook.py	20%
Integração com RabbitMQ	20%
Implementação do worker.py	20%
Atualização correta da Orders API	10%

Competências Avaliadas

- Arquitetura orientada a eventos
- Sistemas distribuídos
- Integração entre microserviços
- Comunicação assíncrona
- Reverse proxy
- Message brokers
- Processamento de eventos
- Integração de APIs
- Webhooks
- Tolerância a falhas
- Observabilidade básica